

```

# We've gotten comfortable with writing functions.

def sumTwo(x, y):
    return x + y

def sumThree(x, y, z):
    return x + y + z

# We know how to have functions call other functions
def sumFour(x, y, z, w):
    return x + y + z + w

# It'd be nice if we could have a function that could sum any number
# of numbers
# How do we do that?
def sumList(nums):
    total = 0
    # ???
    return total

# The typical way we do this is by using a loop:
def sumListFor(nums):
    total = 0
    for num in nums:
        total += num
    return total

# But there's another way we can do this, if we notice an interesting
# pattern about this problem.

# NOTE: if there is a board, do this on the board
# recSumList([0, 1, 2, 3])
# 0 + recSumList([1, 2, 3])
# 0 + 1 + recSumList([2, 3])
# 0 + 1 + 2 + recSumList([3])
# 0 + 1 + 2 + 3
# -> 6

# This is a recursive pattern! We can write a recursive function to do
# this.

```

Recursion Basics

In recursion, a function *calls itself*. Recursion is used when a problem can be easily divided into smaller problems of the same form.

A recursive function has 2 components:

- Base case: Simplest possible input and prevents infinite recursion.
- Recursion step: Call the same function itself with a smaller/easier input to the function and act on the output of the smaller function call.

```

## Anatomy of a Recursive Function

# Recursive functions always have two parts:
# 1. Base Case:
#     - This is the simplest version of the problem that we can solve.
#     - It's the "base" of the recursion.
#     - We always need a base case, otherwise our function will recurse
#       forever!
# 2. Recursive Case:
#     - This is the more complex version of the problem that we can
#       solve.
#     - We always need to make progress towards the base case,
#       otherwise our function will recurse forever!

# Let's try this together for sumList:
def recSumList(nums):
    # What is the "base case" that we know how to solve?
    if len(nums) == 1:
        return nums[0]
    # What is the "recursive case" that we can make progress towards
    # the base case?
    return nums[0] + recSumList(nums[1:])

# Does this work for all lists?
# What happens if we call recSumList([])?
# Let's fix that:
def recSumList(nums):
    # What is the "base case" that we know how to solve?
    if len(nums) == 0:
        return 0
    if len(nums) == 1:
        return nums[0]
    # What is the "recursive case" that we can make progress towards
    # the base case?
    return nums[0] + recSumList(nums[1:])

```

The steps of writing a recursive function:

1. Given an instance of a problem, how can you break the problem down into smaller instances of the same type? (ignore the base-case)
2. What instances of the problem do not follow the above pattern but can be solved using other means? (base case)
3. Put them together

You have to make sure that you are always making progress towards a solution.

We're going to spend the rest of this lecture practicing recognizing recursive problems.

```

# Example one: Count Down from n to 0, then print "Blastoff!"
# What is the base case?

```

```

# What is the recursive case?

# Here's a simpler example of recursion

def recCountdown(n):
    if n == 0:
        print("Blastoff!")
    else:
        print(n)
        recCountdown(n - 1)

# Example Two: Factorial

# Definition of factorial:  $n! = n * (n - 1) * (n - 2) * \dots * 3 * 2 * 1$ 
# Also,  $1! = 1$ 
# How can you turn this into a recursion?
# Recursive case:
# Base case:  $1! = 1$ 
#  $n! = n * (n - 1)!$ 
def factorial(n):
    #n=5
    #n=4
    #n=3
    #n=2
    #n=1
    if n==1:
        return 1
    return n*factorial(n-1)
#factorial(5)
#return 5*factorial(5-1)
    #return 4*factorial(4-1)
#return 3*factorial(3-1)
#return 2*factorial(2-1)
#return 1
print(factorial(5))

def factorial2(n):
    if n == 1:
        return 1
    return n * factorial(n - 1)

print(factorial(1))
print(factorial(2))
print(factorial(3))
print(factorial(4))

# Let's fix the error in a few recursive functions.
# what's wrong here? (talk with someone next to you)

```

```
def count_up(n):  
    """Print out all numbers up to and including n in ascending order."""  
  
    >>> count_up(5)  
    1  
    2  
    3  
    4  
    5  
    """  
    i = 1  
    if i == n:  
        return  
    print(i)  
    i += 1  
    count_up(n-1)  
  
# What does this function do?  
def mystery(a):  
    if a == 0:  
        return 0  
    return a ** mystery(a - 1)  
  
# What is the output of the line of code below?  
mystery(4)
```